OS LAB - Week 11: Producer-Consumer Problem using Semaphores

Name: Aditya Kumar

Registration no: AP24110010333

BTech CSE - C

Question:

Implement a solution to the producer-consumer problem using semaphores.

Approach:

A bounded buffer of size 5 is shared by 3 producers and 3 consumers, and each producer makes 10 items. The program uses pthreads and POSIX semaphores (sem_t) only, with three semaphores doing all of the synchronization. A shared counter holds the number of items in the buffer, and 0 <= counter <= BUFFER_SIZE is checked with assert() inside the critical section.

Semaphore	Initial value	Purpose
empty_slots	5 (BUFFER_SIZE)	Counts free slots. A producer waits on it before inserting, so it blocks when the buffer is full.
full_slots	0	Counts filled slots. A consumer waits on it before removing, so it blocks when the buffer is empty.
mutex	1	Binary semaphore giving mutual exclusion on in, out, counter and the statistics.

```
Producer: wait(empty) -> wait(mutex) -> insert -> post(mutex) -> post(full)
Consumer: wait(full)  -> wait(mutex) -> remove -> post(mutex) -> post(empty)
```

CODE:

```
/*
 * producer_consumer_sem.c
 *
 * Bounded-buffer Producer-Consumer problem solved with POSIX semaphores.
 *
 *   empty_slots : counts free slots in the buffer   (initial value = BUFFER_SIZE)
 *   full_slots  : counts filled slots in the buffer (initial value = 0)
 *   mutex       : binary semaphore protecting in, out and counter (initial value = 1)
 *
 * Producer : wait(empty_slots) -> wait(mutex) -> insert -> post(mutex) -> post(full_slots)
 * Consumer : wait(full_slots)  -> wait(mutex) -> remove -> post(mutex) -> post(empty_slots)
 *
 * Build options (all optional):
 *   -DNUM_PRODUCERS=n  -DNUM_CONSUMERS=n  -DITEMS_PER_THREAD=n
 *   -DSWAP_ORDER=1     take mutex BEFORE empty_slots/full_slots (wrong order, deadlock demo)
 *
 * Compile : gcc -pthread producer_consumer_sem.c -o pc_sem
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <assert.h>

#define BUFFER_SIZE 5

#ifndef NUM_PRODUCERS
#define NUM_PRODUCERS 3
```

```
#endif
#ifndef NUM_CONSUMERS
#define NUM_CONSUMERS 3
#endif
#ifndef ITEMS_PER_THREAD
#define ITEMS_PER_THREAD 10          /* items made by each producer (must be < 100) */
#endif
#ifndef SWAP_ORDER
#define SWAP_ORDER 0                 /* 1 = wrong semaphore order (deadlock demo) */
#endif

#define TOTAL_ITEMS (NUM_PRODUCERS * ITEMS_PER_THREAD)
_Static_assert(ITEMS_PER_THREAD < 100, "item numbering needs ITEMS_PER_THREAD < 100");
_Static_assert(TOTAL_ITEMS % NUM_CONSUMERS == 0, "items must divide evenly among consumers");
#define ITEMS_PER_CONSUMER (TOTAL_ITEMS / NUM_CONSUMERS)

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int counter = 0;                     // number of items currently in the buffer

sem_t empty_slots, full_slots, mutex;

// Statistics and checks (only touched while holding mutex)
int total_produced = 0, total_consumed = 0;
int max_counter_seen = 0;
int next_expected[NUM_PRODUCERS + 1];        // per-producer FIFO check
int seen[(NUM_PRODUCERS + 1) * 100];         // how many times each item was consumed

void *producer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < ITEMS_PER_THREAD; i++) {
        int item = id * 100 + i;

#if SWAP_ORDER
        sem_wait(&mutex);                    // WRONG: holds the mutex while waiting for a
slot
        sem_wait(&empty_slots);
#else
        sem_wait(&empty_slots);              // wait for a free slot
        sem_wait(&mutex);                    // enter critical section
#endif
        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
        total_produced++;
        if (counter > max_counter_seen) max_counter_seen = counter;
        assert(counter >= 0 && counter <= BUFFER_SIZE);
        printf("Producer %d produced item %3d | counter = %d\n", id, item, counter);
        sem_post(&mutex);                    // leave critical section
        sem_post(&full_slots);               // one more filled slot
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < ITEMS_PER_CONSUMER; i++) {
#if SWAP_ORDER
        sem_wait(&mutex);                    // WRONG: holds the mutex while waiting for an
item
        sem_wait(&full_slots);
```

```
#else
        sem_wait(&full_slots);              // wait for a filled slot
        sem_wait(&mutex);                   // enter critical section
#endif
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        counter--;
        total_consumed++;
        assert(counter >= 0 && counter <= BUFFER_SIZE);

        int p = item / 100, idx = item % 100;
        assert(idx == next_expected[p]);    // each producer's items leave in FIFO order
        next_expected[p]++;
        seen[item]++;

        printf("Consumer %d consumed item %3d | counter = %d\n", id, item, counter);
        sem_post(&mutex);                   // leave critical section
        sem_post(&empty_slots);             // one more free slot
    }
    return NULL;
}

int main(void) {
    pthread_t prod[NUM_PRODUCERS], cons[NUM_CONSUMERS];
    int pid[NUM_PRODUCERS], cid[NUM_CONSUMERS];

    setvbuf(stdout, NULL, _IOLBF, 0);       // line-buffered, so output survives a hang
    sem_init(&empty_slots, 0, BUFFER_SIZE);
    sem_init(&full_slots, 0, 0);
    sem_init(&mutex, 0, 1);

    printf("Semaphores : empty_slots = %d, full_slots = 0, mutex = 1\n", BUFFER_SIZE);
    printf("Wait order : %s\n", SWAP_ORDER ? "mutex first (WRONG, deadlock demo)"
                                           : "empty/full first, then mutex (correct)");
    printf("Config     : %d producer(s), %d consumer(s), %d items per producer, buffer size
%d\n\n",
           NUM_PRODUCERS, NUM_CONSUMERS, ITEMS_PER_THREAD, BUFFER_SIZE);

    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pid[i] = i + 1;
        pthread_create(&prod[i], NULL, producer, &pid[i]);
    }
    for (int i = 0; i < NUM_CONSUMERS; i++) {
        cid[i] = i + 1;
        pthread_create(&cons[i], NULL, consumer, &cid[i]);
    }
    for (int i = 0; i < NUM_PRODUCERS; i++) pthread_join(prod[i], NULL);
    for (int i = 0; i < NUM_CONSUMERS; i++) pthread_join(cons[i], NULL);

    int once = 1;                           // every item consumed exactly once?
    for (int p = 1; p <= NUM_PRODUCERS; p++)
        for (int i = 0; i < ITEMS_PER_THREAD; i++)
            if (seen[p * 100 + i] != 1) once = 0;

    int ev, fv, mv;
    sem_getvalue(&empty_slots, &ev);
    sem_getvalue(&full_slots, &fv);
    sem_getvalue(&mutex, &mv);

    printf("\nTotal produced = %d\n", total_produced);
    printf("Total consumed = %d\n", total_consumed);
```

```
    printf("Max counter observed = %d (limit %d)\n", max_counter_seen, BUFFER_SIZE);
    printf("Every item consumed exactly once, in FIFO order per producer : %s\n", once ? "yes"
: "NO");
    printf("Final semaphore values : empty_slots = %d, full_slots = %d, mutex = %d\n", ev, fv,
mv);
    printf("\nFinal counter = %d (expected 0)\n", counter);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    sem_destroy(&mutex);
    return (counter == 0 && once && total_produced == total_consumed) ? 0 : 1;
}
```

OUTPUT:

For case-1: 3 producers, 3 consumers, 10 items per producer, buffer size 5 (correct order)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -Wall -Wextra producer_consumer_sem.c -o
pc_sem
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_sem
Semaphores : empty_slots = 5, full_slots = 0, mutex = 1
Wait order : empty/full first, then mutex (correct)
Config     : 3 producer(s), 3 consumer(s), 10 items per producer, buffer size 5

Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 2
Producer 1 produced item 102 | counter = 3
Producer 1 produced item 103 | counter = 4
Producer 1 produced item 104 | counter = 5
Consumer 1 consumed item 100 | counter = 4
Consumer 1 consumed item 101 | counter = 3
Consumer 1 consumed item 102 | counter = 2
Consumer 1 consumed item 103 | counter = 1
Consumer 1 consumed item 104 | counter = 0
Producer 3 produced item 300 | counter = 1
Producer 3 produced item 301 | counter = 2
Producer 3 produced item 302 | counter = 3
Producer 3 produced item 303 | counter = 4
Producer 3 produced item 304 | counter = 5
Consumer 2 consumed item 300 | counter = 4
Consumer 2 consumed item 301 | counter = 3
Consumer 2 consumed item 302 | counter = 2
Consumer 2 consumed item 303 | counter = 1
Consumer 2 consumed item 304 | counter = 0
Producer 1 produced item 105 | counter = 1
Producer 1 produced item 106 | counter = 2
Producer 1 produced item 107 | counter = 3
Producer 1 produced item 108 | counter = 4
Producer 1 produced item 109 | counter = 5
Consumer 3 consumed item 105 | counter = 4
Consumer 3 consumed item 106 | counter = 3
Consumer 3 consumed item 107 | counter = 2
Consumer 3 consumed item 108 | counter = 1
Consumer 3 consumed item 109 | counter = 0
Producer 2 produced item 200 | counter = 1
Producer 2 produced item 201 | counter = 2
Producer 2 produced item 202 | counter = 3
Producer 2 produced item 203 | counter = 4
Producer 2 produced item 204 | counter = 5
Consumer 1 consumed item 200 | counter = 4
Consumer 1 consumed item 201 | counter = 3
Consumer 1 consumed item 202 | counter = 2
```

```
Consumer 1 consumed item 203 | counter = 1
Consumer 1 consumed item 204 | counter = 0
Producer 3 produced item 305 | counter = 1
Producer 3 produced item 306 | counter = 2
Producer 3 produced item 307 | counter = 3
Producer 3 produced item 308 | counter = 4
Producer 3 produced item 309 | counter = 5
Consumer 2 consumed item 305 | counter = 4
Consumer 2 consumed item 306 | counter = 3
Consumer 2 consumed item 307 | counter = 2
Consumer 2 consumed item 308 | counter = 1
Consumer 2 consumed item 309 | counter = 0
Producer 2 produced item 205 | counter = 1
Producer 2 produced item 206 | counter = 2
Producer 2 produced item 207 | counter = 3
Producer 2 produced item 208 | counter = 4
Producer 2 produced item 209 | counter = 5
Consumer 3 consumed item 205 | counter = 4
Consumer 3 consumed item 206 | counter = 3
Consumer 3 consumed item 207 | counter = 2
Consumer 3 consumed item 208 | counter = 1
Consumer 3 consumed item 209 | counter = 0

Total produced = 30
Total consumed = 30
Max counter observed = 5 (limit 5)
Every item consumed exactly once, in FIFO order per producer : yes
Final semaphore values : empty_slots = 5, full_slots = 0, mutex = 1

Final counter = 0 (expected 0)
```

For case-2: 1 producer, 1 consumer (correct order)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -DNUM_PRODUCERS=1 -DNUM_CONSUMERS=1
producer_consumer_sem.c -o pc_sem_1p1c
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_sem_1p1c
Semaphores : empty_slots = 5, full_slots = 0, mutex = 1
Wait order : empty/full first, then mutex (correct)
Config     : 1 producer(s), 1 consumer(s), 10 items per producer, buffer size 5

Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 2
Producer 1 produced item 102 | counter = 3
Producer 1 produced item 103 | counter = 4
Producer 1 produced item 104 | counter = 5
Consumer 1 consumed item 100 | counter = 4
Consumer 1 consumed item 101 | counter = 3
Consumer 1 consumed item 102 | counter = 2
Consumer 1 consumed item 103 | counter = 1
Producer 1 produced item 105 | counter = 2
Producer 1 produced item 106 | counter = 3
Consumer 1 consumed item 104 | counter = 2
Producer 1 produced item 107 | counter = 3
Consumer 1 consumed item 105 | counter = 2
Producer 1 produced item 108 | counter = 3
Consumer 1 consumed item 106 | counter = 2
Producer 1 produced item 109 | counter = 3
Consumer 1 consumed item 107 | counter = 2
Consumer 1 consumed item 108 | counter = 1
Consumer 1 consumed item 109 | counter = 0
```

```
Total produced = 10
Total consumed = 10
Max counter observed = 5 (limit 5)
Every item consumed exactly once, in FIFO order per producer : yes
Final semaphore values : empty_slots = 5, full_slots = 0, mutex = 1

Final counter = 0 (expected 0)
```

For case-3: semaphore order swapped (mutex taken first), 1 producer, 1 consumer, 50 items: deadlock

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -DSWAP_ORDER=1 -DNUM_PRODUCERS=1 -
DNUM_CONSUMERS=1 -DITEMS_PER_THREAD=50 producer_consumer_sem.c -o pc_sem_deadlock
aditya-forge@LAPTOP-184D82EB:~/os_lab$ timeout 5 ./pc_sem_deadlock
Semaphores : empty_slots = 5, full_slots = 0, mutex = 1
Wait order : mutex first (WRONG, deadlock demo)
Config     : 1 producer(s), 1 consumer(s), 50 items per producer, buffer size 5

Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 2
Producer 1 produced item 102 | counter = 3
Producer 1 produced item 103 | counter = 4
Producer 1 produced item 104 | counter = 5
aditya-forge@LAPTOP-184D82EB:~/os_lab$ echo $?
124
```

Correct build: all assertions held, the counter never left the range 0 to 5, every item was consumed exactly once (in FIFO order per producer), the semaphores returned to their starting values and the final counter was 0. This was repeated 200 times with 3 producers and 3 consumers and 100 times with 1 producer and 1 consumer, and every run passed. The swapped-order build (case-3) deadlocked in 20 of 20 runs.

REPORT:

1. Role of the semaphores

empty_slots and full_slots are counting semaphores that decide when a thread may proceed: a producer sleeps while the buffer is full and a consumer sleeps while it is empty, with no polling. mutex is a binary semaphore that makes the update of in, out and counter one indivisible step.

2. Why mutex is needed as well

empty_slots and full_slots only say that a slot is available. With several producers, two of them can pass wait(empty_slots) at the same moment and then write the same slot or both run counter++ and lose an update, which is the race seen in the shared-memory assignment. The mutex serializes the short critical section. It is held only for the insert or remove, never while waiting for a slot.

3. Why the order of wait() matters

In case-3 the order is swapped so that each thread takes mutex first. Depending on timing, the first thread to run out of what it needs goes to sleep while still holding mutex: a producer facing a full buffer waits on empty_slots, or a consumer facing an empty buffer waits on full_slots. The other side needs mutex to make progress (a consumer to free a slot, or a producer to add an item), so it blocks on mutex and nobody can ever continue (deadlock). In the run shown above the producer filled the buffer (5 items made, 0 taken), then slept on empty_slots while holding mutex, so the consumer could never get in to take an item. The program hung in 20 of 20 runs and was stopped by timeout (exit status 124). The correct order waits on the counting semaphore first, while holding nothing, and takes mutex only afterwards.

4. Difference from a spinlock solution

With semaphores a thread that cannot proceed is put to sleep by the OS and woken by the matching post(), so it does not burn CPU. A test_and_set spinlock keeps the CPU busy while waiting, and the full/empty condition had to be re-tested in a retry loop with sched_yield(). Here the counting semaphores replace that polling.

5. Correctness checks used

(1) assert(0 <= counter <= BUFFER_SIZE) inside every critical section; (2) each producer's items are removed in the order they were produced; (3) every produced item is consumed exactly once; (4) total produced equals total consumed; (5) final counter is 0 and the semaphores are back at empty_slots = 5, full_slots = 0, mutex = 1.